

Question Bank — Week 11

Trees (Lecture 11) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the code	5
Part D — Tree state — draw every step	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	7
Part G — Write the code	8

Answers are in a separate file: [week11-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. `BinarySearchTree` and `TreeNode` are the classes of `dsa/tree.py`: values are unique (inserting a duplicate changes nothing), **height** counts **edges** (empty tree -1 , one node 0), and `delete` of a node with two children uses the **in-order successor**, the smallest value in its right subtree. The four traversals return lists. Unless a question says otherwise, **Q** is the BST built by inserting, in this order, into an empty tree:

`Q = [30, 15, 45, 8, 20, 40, 60, 17, 25, 50]`

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W11-M01 – W11-M22
B	Short answer and essay	W11-E1 – W11-E5
C	Trace the code	W11-T1 – W11-T5
D	Tree state — draw every step	W11-S1 – W11-S3
E	Complexity analysis	W11-K1 – W11-K3
F	Find and fix the bug	W11-B1 – W11-B4
G	Write the code — checked by <code>pytest</code>	W11-C1 – W11-C5

Part A — Multiple choice

W11-M01 [what] A **leaf** is a node that:

- **a)** has no parent
- **b)** has no children
- **c)** has exactly one child
- **d)** is at depth 1

W11-M02 [what] With height counted in edges, the height of a tree with exactly one node is:

- **a)** -1
- **b)** 1
- **c)** 0
- **d)** undefined

W11-M03 [how] The largest number of nodes a binary tree of height 3 can hold is:

- **a)** 7
- **b)** 8
- **c)** 16
- **d)** 15

W11-M04 [how] The smallest possible height of a binary tree with 100 nodes is:

- **a)** 6
- **b)** 7
- **c)** 10
- **d)** 99

W11-M05 [what] A binary tree is a binary **search** tree when, for every node x :

- **a)** x 's left child is smaller than x and its right child is larger
- **b)** every value in x 's left subtree is smaller than x , and every value in its right subtree is larger
- **c)** x is smaller than both of its children
- **d)** the values read level by level are sorted

W11-M06 [how] In the tree Q , the **right child of the root's left child** is:

- **a)** 20
- **b)** 25
- **c)** 17
- **d)** 8

W11-M07 [how] `contains(25)` on Q compares 25 with how many nodes?

- **a)** 3
- **b)** 5
- **c)** 4
- **d)** 10

W11-M08 [why] The worst-case cost of contains on a BST of n nodes, built by `dsa/tree.py` from values in an unknown order, is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$

W11-M09 [how] Which insertion order of 1 to 7 builds the **tallest** BST?

- **a)** 4, 2, 6, 1, 3, 5, 7
- **b)** 4, 6, 5, 7, 2, 3, 1
- **c)** 3, 1, 2, 6, 5, 7, 4
- **d)** 7, 1, 6, 2, 5, 3, 4

W11-M10 [how] The **pre-order** walk of Q is:

- **a)** 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
- **b)** 30, 15, 8, 20, 17, 25, 45, 40, 60, 50
- **c)** 30, 15, 45, 8, 20, 40, 60, 17, 25, 50
- **d)** 8, 17, 25, 20, 15, 40, 50, 60, 45, 30

W11-M11 [how] The **post-order** walk of Q is:

- **a)** 8, 17, 25, 20, 15, 40, 50, 60, 45, 30
- **b)** 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
- **c)** 30, 45, 60, 50, 40, 15, 20, 25, 17, 8
- **d)** 17, 25, 8, 20, 15, 50, 40, 60, 45, 30

W11-M12 [what] Which walk of **any** BST lists its values in increasing order?

- **a)** pre-order
- **b)** post-order
- **c)** in-order
- **d)** level-order

W11-M13 [why] You save a BST to a file as a list of values, and later rebuild it by inserting the list into an empty tree. To get back **the same shape**, the list should be the tree's:

- **a)** in-order walk
- **b)** pre-order walk
- **c)** post-order walk
- **d)** in-order walk, reversed

W11-M14 [why] Which walk frees a tree safely in a language where you release memory yourself, and evaluates an expression tree?

- **a)** post-order
- **b)** pre-order
- **c)** in-order
- **d)** level-order

W11-M15 [why] `level_order` uses a queue rather than the call stack because:

- **a)** a queue uses less memory than recursion
- **b)** it must visit nodes in order of distance from the root, and the call stack always goes deeper into the most recent node first
- **c)** recursion cannot visit every node of a tree
- **d)** the tree stores its nodes in a queue

W11-M16 [how] During `level_order` on `Q`, the largest number of nodes waiting in the queue at one time is:

- **a)** 2
- **b)** 3
- **c)** 4
- **d)** 10

W11-M17 [what] Deleting a node with **two** children, `dsa/tree.py` replaces its value with:

- **a)** the largest value in its right subtree
- **b)** the value of its right child
- **c)** the value of its parent
- **d)** the smallest value in its right subtree

W11-M18 [how] `delete(30)` on `Q`. Which value ends up at the root?

- **a)** 40
- **b)** 25
- **c)** 45
- **d)** 17

W11-M19 [why] The in-order successor of a node with two children never has a left child. Why does that matter to `delete`?

- **a)** it makes the successor the largest value in the tree
- **b)** removing the successor is then always a leaf or one-child case, never a second two-children case
- **c)** it means the successor is always a leaf
- **d)** it lets `delete` skip the search

W11-M20 [why] A student's `is_valid` checks, at every node, only that the left child is smaller and the right child larger. It:

- **a)** accepts some trees that are not BSTs
- **b)** rejects some valid BSTs
- **c)** is correct, but $O(n^2)$
- **d)** is correct and $O(n)$

W11-M21 [how] The expression tree for $(2 + 3) * 4$ is walked in post-order. The result is:

- **a)** `* + 2 3 4`
- **b)** `2 + 3 * 4`
- **c)** `2 3 + 4 *`
- **d)** `2 3 4 + *`

W11-M22 [what] AVL trees and red-black trees:

- **a)** are the BSTs of `dsa/tree.py`, under other names
 - **b)** store their nodes in an array, like a heap
 - **c)** are faster than hash tables for every operation
 - **d)** re-shape themselves on insert and delete so that the height stays $O(\log n)$
-

Part B — Short answer and essay

W11-E1 [what] (3 marks) Define root, leaf, parent, subtree, depth and height, using the tree Q as the example for each. Why does a tree of n nodes always have exactly $n - 1$ edges?

W11-E2 [why] (4 marks) State the BST property. Give a tree in which every parent-child pair is in the right order but which is **not** a BST, and explain how `is_valid` must be designed to reject it. What is its cost?

W11-E3 [why] (4 marks) Describe the three cases of BST deletion. For the two-children case, explain why the in-order successor may take the deleted value's place, and why removing the successor is always easy.

W11-E4 [why] (4 marks) Name the four traversals of a binary tree, give the order of each, and for each say one thing it is **used for** and why it suits that use.

W11-E5 [why] (4 marks) "A BST gives $O(\log n)$ search." When is that true, and when is it false? Explain with the height, give an insertion order that makes it false, and name the cure. Compare a BST with a sorted array for search, insert and listing in order.

Part C — Trace the code

W11-T1 [how] Trace `contains(25)` and `contains(42)` on Q: list each node compared, the decision taken, and the result. Then say where `insert(42)` would put the new node.

W11-T2 [how] Give the four walks — pre-order, in-order, post-order and level-order — of the BST built from `[12, 5, 18, 2, 9, 15, 20, 7, 16]`.

W11-T3 [why] (a) The **pre-order** walk of a BST is `[20, 10, 5, 15, 12, 30, 25, 40]`. Draw the tree and give its post-order walk. (b) A binary tree that is **not** a BST has pre-order `A B D E C F` and in-order `D B E A C F`. Draw it and give its post-order walk. Why does (b) need two walks when (a) needed one?

W11-T4 [how] What do these functions return for the root of Q?

```
def f(node):  
    if node is None:  
        return 0  
    return node.value + max(f(node.left), f(node.right))
```

```
def g(node, k):  
    if node is None:  
        return 0  
    if k == 0:  
        return 1  
    return g(node.left, k - 1) + g(node.right, k - 1)
```

Give $f(\text{root})$, and $g(\text{root}, k)$ for $k = 0, 1, 2, 3, 4$. Say in words what each function computes.

W11-T5 [why] Trace `_is_valid(node, low, high)` of `dsa/tree.py` on the hand-built tree below: for each call, the node, low and high, and what it returns. Which call first returns False?

```
root = TreeNode(20, TreeNode(10, TreeNode(5), TreeNode(25)), TreeNode(30))
```

Part D — Tree state — draw every step

W11-S1 [how] Insert 45, 20, 70, 10, 30, 60, 90, 25, 35, 65 into an empty BST, drawing the tree after each insert and giving its height. Then, on the final tree, perform `delete(90)`, `delete(70)`, `delete(45)` and `delete(20)`, in that order, drawing the tree after each. For each delete, name the case, and for the two-children cases name the successor.

W11-S2 [how] Draw the BST built by each insertion order of the values 1 to 7, and give its height:

- (a) 1, 2, 3, 4, 5, 6, 7
- (b) 4, 2, 6, 1, 3, 5, 7
- (c) 3, 1, 2, 6, 5, 7, 4
- (d) 4, 6, 5, 7, 2, 3, 1

Which two give the same tree? Give their pre-order walks, and explain why they must match.

W11-S3 [how] Draw the queue of `level_order` (front on the left) after every dequeue, for the tree Q. Then do the same walk with a **stack** in place of the queue, pushing the **right** child before the left, and give the order in which the values come out. Which of the named walks is it?

Part E — Complexity analysis

W11-K1 [how] Give Θ and justify:

- (a) building a BST by inserting the values 1, 2, ..., n in order;
- (b) the same n values inserted in a random order (on average);
- (c) **tree sort**: insert all n values, then walk in-order — in the best and in the worst case.

W11-K2 [why] A student writes height like this:

```
def height(node):  
    if node is None:  
        return -1  
    if height(node.left) > height(node.right):  
        return 1 + height(node.left)  
    return 1 + height(node.right)
```

It returns the right answer. Write a recurrence for the number of calls it makes on a chain of n nodes that all hang to the **right**, and on a perfect tree of height h. Solve both. How many calls does it make on a right chain of 20 nodes? What is the fix?

W11-K3 [why] Compare the **extra memory** of the recursive in-order walk and of level_order: on a perfect tree of n nodes, and on a chain of n nodes. Which is better on which tree, and why?

Part F — Find and fix the bug

W11-B1 [how]

```
def insert(self, value):  
    node = self.root  
    while node is not None:  
        if value == node.value:  
            return  
        node = node.left if value < node.value else node.right  
    node = TreeNode(value)
```

W11-B2 [why]

```
def is_valid(self):  
    return self._ok(self.root)  
  
def _ok(self, node):  
    if node is None:  
        return True  
    if node.left is not None and node.left.value >= node.value:  
        return False  
    if node.right is not None and node.right.value <= node.value:  
        return False  
    return self._ok(node.left) and self._ok(node.right)
```

W11-B3 [why] The two-children part of a delete, after node (with parent parent) has been found:

```
if node.left is not None and node.right is not None:
    succ_parent, succ = node, node.right
    while succ.left is not None:
        succ_parent, succ = succ, succ.left
    node.value = succ.value
    succ_parent.left = succ.right
    return
```

It passes `test_delete_node_with_two_children`. Find an input on which it breaks the tree, and fix it.

W11-B4 [how]

```
def in_order(self):
    out = []
    self._in_order(self.root, out)
    return out

def _in_order(self, node, out):
    out = []
    if node is not None:
        self._in_order(node.left, out)
        out.append(node.value)
        self._in_order(node.right, out)
```

Part G — Write the code

In `practice/week11.py`; check with `pytest tests/test_practice_week11.py -v`. Each function takes a tree by its **root** — a `TreeNode`, or `None` — not a `BinarySearchTree`. The tests hand you nodes that count how often their `value`, `left` and `right` are read, so an answer that visits more of the tree than it needs fails. Do not copy the tree into a list and work on the list.

W11-C1 [how] `count_leaves(node)` — the number of leaves.

W11-C2 [why] `is_balanced(node)` — True when at **every** node the two subtrees' heights differ by at most 1, in **one** $O(n)$ walk. Why is calling a separate height at every node not good enough?

W11-C3 [why] `range_values(node, low, high)` — the values v with $low \leq v \leq high$, in sorted order, visiting only the part of the tree that can hold them. Which subtrees may you skip, and why?

W11-C4 [how] `build_balanced(values)` — from a sorted list of distinct values, a BST of

the smallest possible height. No slicing.

W11-C5 [why] `lowest_common_ancestor(node, a, b)` — in a BST, the lowest node with both a and b in its subtree, in $O(h)$ with a loop. Why is the first node where a and b “split” the answer?